MBLP131 · HAFTA 2

İşletim Sistemleri Temelleri

Hafta 2 · Tanımlar, Hedefler, Bileşenler ve İşletim Sistemi Yapıları

İşletim sistemi tam olarak nedir, ne işe yarar ve nasıl inşa edilir?

Elektronik Haberleşme Teknolojisi Programı · Meslek Yüksekokulu · İstanbul Okan Üniversitesi

GEÇEN HAFTA

Dört Fikirde 1. Hafta

Bilgisayar komut izler

Girdi → işlem → çıktı; program, verinin yanında bellekte saklanır.

İlk o başlar

İşletim sistemi ilk başlar ve diğer programlar çalışabilsin diye her şeyi yönetir.

Boş işlemci israftır

Toplu işleme, çoklu programlama ve zaman paylaşımı, işlemciyi meşgul tutmak için icat edildi.

UNIX 1969, Linux 1991

Küçük, birleştirilebilir araçlar; 9. haftadan itibaren uygulamalı olarak Linux kullanacağız.

Bugün gayriresmî tanımın yerine kesin bir tanım koyuyor ve işletim sistemlerinin gerçekte nasıl inşa edildiğine bakıyoruz.

Bugünün Gündemi

1. **Tanımlar** İşletim sistemi tam olarak nedir? İki klasik bakış açısı.
2. **Hedefler** Kullanım kolaylığı, verimlilik, gelişebilirlik ve aralarındaki ödünleşimler.
3. **Bileşenler** Donanım, işletim sistemi, uygulamalar, kullanıcılar; kesmeler, bellek, ikili mod.
4. **Yapılar** Basit, monolitik, katmanlı, mikroçekirdek, modüler, hibrit.
5. **Uygulama** Gerçek C kodu ve strace ile sistem çağrılarını izlemek.

BÖLÜM 1

İşletim Sistemi Tam Olarak Nedir?

Gayriresmî bir tariften kesin bir tanıma.

TANIMLAR

Karşınıza Çıkacak Üç Tanım

Aracı

Bilgisayarın kullanıcısı ile bilgisayar donanımı arasında aracılık eden (intermediary) bir program.

— Silberschatz, Galvin ve Gagne

Çekirdek

Bilgisayarda her an çalışan tek program: çekirdek (kernel). Geri kalan her şey ya bir sistem programı ya da bir uygulama programıdır.

— Silberschatz, Galvin ve Gagne

Yönetici

Programların yürütülmesini denetleyen ve bilgisayarın kaynaklarını (işlemci, bellek, dosyalar, aygıtlar) yöneten yazılım.

— Stallings (özetle)

Bu dersteki çalışma tanımımız: işletim sistemi, donanımı yöneten ve programlara üzerinde çalışabilecekleri daha basit, daha güvenli bir makine sunan yazılım katmanıdır. Bellekte sürekli bulunan özüne çekirdek (kernel) denir.

Genişletilmiş Makine ve Kaynak Yöneticisi

Andrew Tanenbaum işletim sistemini iki yönden anlatır: yukarıdan aşağıya ve aşağıdan yukarıya.

Yukarıdan aşağıya: genişletilmiş makine

- Karmaşık donanımı temiz soyutlamaların (abstraction) arkasına gizler.
- Disk sektörlerini ve manyetik izleri değil, dosyaları görürsünüz.
- Kesme hatlarını ve yazmaçları değil, süreçleri görürsünüz.
- Hedef: bilgisayarı programlamayı ve kullanmayı kolaylaştırmak.

Aşağıdan yukarıya: kaynak yöneticisi

- Birçok program ve kullanıcı tek bir makine için yarışır.
- **Zaman çoğullama (time multiplexing):** sırayla kullanım (işlemci, yazıcı).
- **Alan çoğullama (space multiplexing):** parçalara bölerek paylaşım (bellek, disk alanı).
- Hedef: kaynakları adil, verimli ve güvenli biçimde dağıtmak.

İki bakış da aynı yazılımı anlatır. Bu dersin 3–7. haftaları çoğunlukla kaynak yöneticisi bakışıdır: süreçler, işlemci, bellek, dosyalar.

TANIMLAR

Çekirdek, Sistem Programları ve Uygulamalar

İşletim sistemiyle birlikte gelen her şey "çekirdek" değildir.



Her katman yalnızca altındaki katmanın hizmetlerini kullanır.

Kim kimdir?

- **Çekirdek:** her zaman bellektedir, ayrıcalıklı bir modda çalışır, donanımla doğrudan konuşur.
- **Sistem programları:** işletim sistemiyle gelir ama normal program olarak çalışır; sistemi kullanılabilir kılar.
- **Uygulamalar:** asıl işinizi yapmak için kurduğunuz her şey.
- Linux, tam anlamıyla yalnızca çekirdektir; bir "Linux dağıtımı" buna (çoğu GNU'dan gelen) sistem programlarını ekler.

TARTIŞMA

Bu, İşletim Sisteminin Parçası mı?

Tek bir resmî yanıt yok; asıl mesele de bu. Önce oylayın, sonra tartışın.

Web tarayıcısı?

1998'de ABD hükümeti Microsoft'a, kısmen Internet Explorer'ı Windows'a gömdüğü için dava açtı. Tarayıcı "Windows'un parçası" mı? Avukatlar bunu yıllarca tartıştı.

Grafik masaüstü?

Windows'ta sisteme sıkıca bütünleşiktir. Linux'ta ise çekirdeğin üzerinde çalışan, değiştirilebilir bir programdır (GNOME, KDE, ...).

Uygulama mağazası?

Telefonda çoğu kullanıcı uygulama mağazasını "sistemin kendisi" sanır. Teknik olarak özel izinlere sahip bir uygulamadır.

Kullanışlı bir kural: çekirdek kesinlikle işletim sistemidir. Çekirdeğin ötesinde "işletim sistemi", üreticinin standart sistem olarak sunduğu her şeydir.

BÖLÜM 2

İşletim Sistemi Ne İşe Yarar?

Üç tasarım hedefi ve bu hedeflerin neden farklı yönlere çektiği.

TASARIM HEDEFLERİ

Her İşletim Sisteminin Üç Hedefi

William Stallings bir işletim sisteminin amaçlarını üç kelimeyle özetler.

1. Kullanım kolaylığı

- Bilgisayarı kullanmayı ve programlamayı kolaylaştırır.
- Dosyalar, pencereler, kabuk, basit sistem çağrıları.
- Kullanıcıların donanımı bilmesi gerekmez.

2. Verimlilik

- Donanımı iyi kullanır: işlemci meşgul, bellek yararlı işle dolu, diskler beklemiyor.
- Düşük ek yük (overhead): işletim sisteminin kendisi az kaynak tüketmeli.

3. Gelişebilirlik

- Yeni donanım (SSD, GPU, 5G modem) ve yeni hizmetler eklenebilmeli…
- …mevcut programlar bozulmadan.
- İşletim sistemi yapısının seçimini belirler (Bölüm 4).

İleride ele alacağımız her yapı, bu üç hedef arasında farklı bir uzlaşmadır.

Hangi Hedef Daha Önemli? Makineye Bağlı

Sistem türü	Ana öncelik	Neden
Kişisel bilgisayar	Kullanım kolaylığı	Her şeyin "öylece çalışmasını" isteyen tek bir kullanıcı; işlemci zaten çoğu zaman boştadır.
Sunucu / bulut	Verimlilik	Binlerce kullanıcı; %1 daha fazla iş hacmi, yüzlerce makine daha az demek olabilir.
Akıllı telefon	Verimlilik (enerji)	Pil ömrü: işletim sistemi uygulamaları ve işlemciyi agresif biçimde uyutur.
Gömülü / IoT	Verimlilik (boyut, maliyet)	Bir çamaşır makinesi denetleyicisinin birkaç kilobaytlık belleği olabilir; hiç işletim sistemi olmayabilir.
Gerçek zamanlı (araç frenleri, dronlar)	Öngörülebilirlik	Geç gelen yanıt yanlış yanıttır: son tarihler (deadline) her zaman karşılanmalıdır.

Kolaylık ve Verimlilik: Çözümlü Örnek

Durum

- Bir program bir dosyadan 1 milyon sayı okumalıdır.
- **Kolay yol:** işletim sisteminden her seferinde bir sayı istemek → 1 milyon sistem çağrısı.
- **Verimli yol:** 4096 baytlık bloklar istemek (her biri ≈ 1000 sayı) → yaklaşık 1000 sistem çağrısı.
- Bir sistem çağrısının saf ek yükü yaklaşık 0,1 µs ise (bunu bugün ölçeceğiz):

```
teker teker:
  1 000 000 × 0.1 µs = 100 ms

1000'lik bloklarla:
      1 000 × 0.1 µs = 0.1 ms

=> 1000 kat daha az ek yük
```

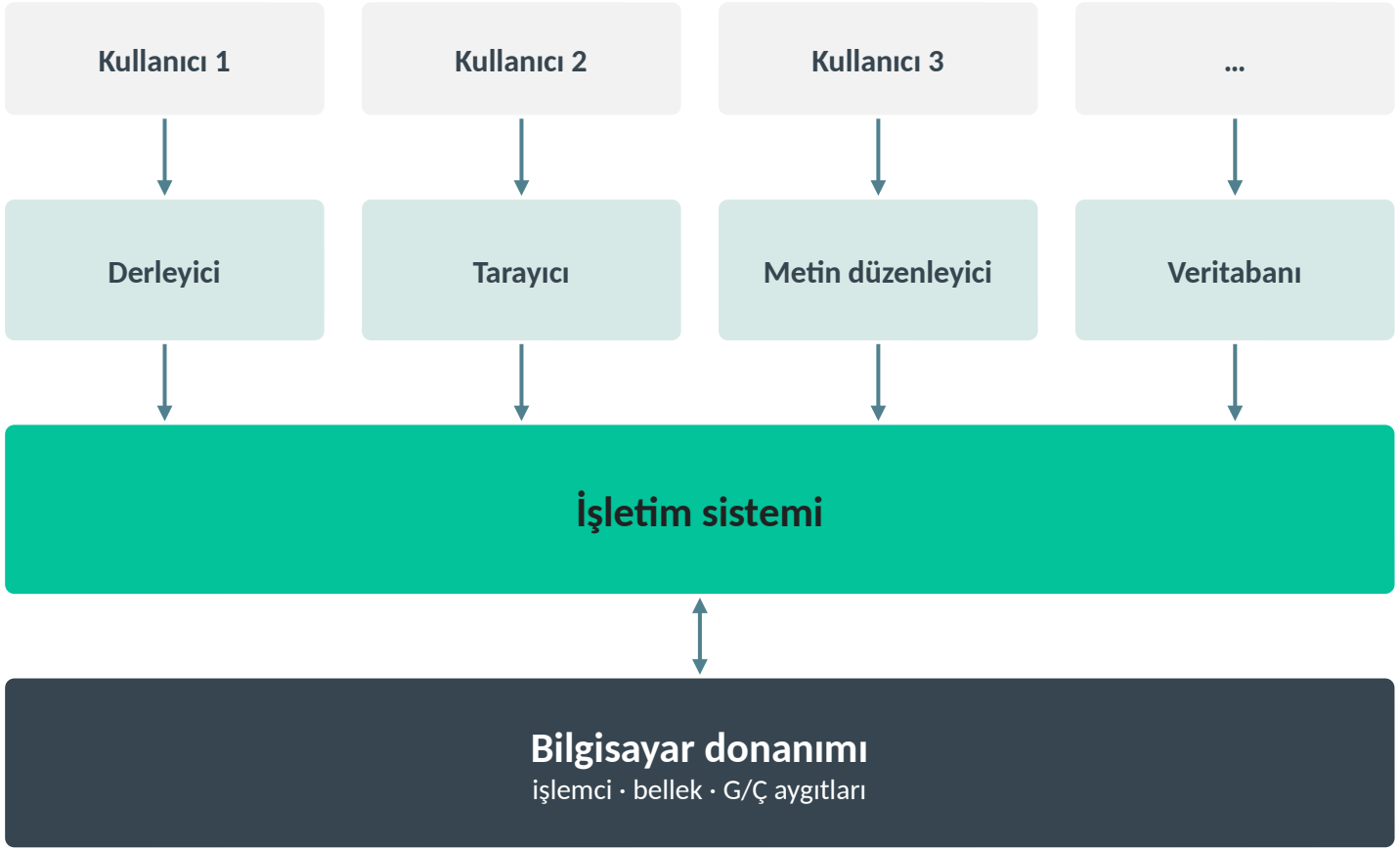
C'nin stdio gibi kütüphaneleri ikisini birden verir: fgetc() fonksiyonunu karakter karakter çağırırsınız, ama kütüphane arka planda büyük bloklar okur.

BÖLÜM 3

Bilgisayar Sisteminin Bileşenleri

Donanım, işletim sistemi, uygulama programları ve kullanıcılar.

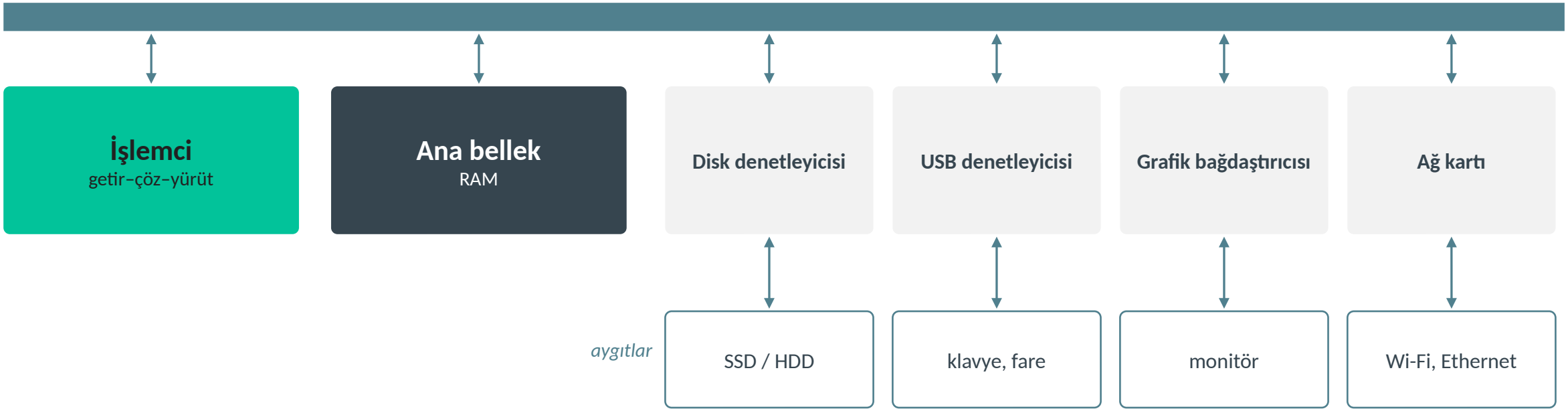
Bilgisayar Sisteminin Dört Bileşeni



Roller

- **Donanım** temel hesaplama kaynaklarını sağlar.
- **Uygulama programları** bu kaynakları kullanıcıların sorunlarını çözmek için kullanır.
- **Kullanıcılar:** insanlar, ama aynı zamanda başka makineler ve programlar.
- **İşletim sistemi** donanımı denetler ve onun uygulamalar ile kullanıcılar arasında kullanımını eşgüdümler.

Kasanın İçi: Yol Üzerinde İşlemci, Bellek ve Aygıtlar



Her aygıt denetleyicisinin küçük bir yerel tamponu vardır ve işlemciyle paralel çalışır. İşletim sistemi her denetleyici için bir aygıt sürücüsü (device driver) içerir: sürücü denetleyicinin dilini konuşur, böylece işletim sisteminin geri kalanının bunu bilmesi gerekmez.

Kesmeler: Aygıt İşlemcinin Dikkatini Nasıl Çeker?

İşlemci klavyeye sürekli "yeni bir şey var mı?" diye sormaz; klavye işlemciye haber verir.

1. Aygıt bitirir

Disk denetleyicisi veriyi kendi tamponuna kopyalamıştır.

→

2. Sinyal

İşlemcinin bir kesme (interrupt) hattını etkinleştirir.

→

3. Kaydet ve atla

İşlemci yaptığı işi kaydeder ve kesme vektöründe yazılı işleyiciye atlar.

→

4. İşle

İşletim sisteminin kesme işleyicisi (sürücünün parçası) olayı işler.

→

5. Devam et

Kaydedilen durum geri yüklenir; kesilen program hiçbir şey olmamış gibi sürer.

Modern işletim sistemleri kesme güdümlüdür (interrupt-driven): çalışan bir program yoksa ve hiçbir aygıt ilgi beklemiyorsa işletim sistemi yalnızca bekler. Her tuş vuruşu, her ağ paketi ve biten her disk okuması bir kesme olarak gelir; saniyede binlerce kez.

Bellek Hiyerarşisi: Hızlı ve Küçük, Yavaş ve Büyük

Düzey	Tipik boyut	Tipik erişim süresi	Yazmaç erişimi 1 saniye sürseydi...
İşlemci yazmaçları	< 1 KB	≈ 0,3 ns	1 saniye
Önbellek (L1–L3)	birkaç MB	1 – 40 ns	3 saniye – 2 dakika
Ana bellek (RAM)	8 – 64 GB	≈ 100 ns	≈ 5 dakika
SSD	0,5 – 4 TB	≈ 50 µs	≈ 2 gün
Sabit disk (HDD)	1 – 20 TB	≈ 5 ms	≈ 6 ay

Yazmaçlar, önbellek ve RAM uçucudur (güç kesilince silinir); SSD ve diskler kalıcıdır. İşletim sistemi verileri düzeyler arasında sürekli taşır; bu, 6. ve 7. haftaların konusudur.

İkili Mod: Kullanıcı Modu ve Çekirdek Modu

Kimin neyi yapabileceğini, işlemcinin içindeki bir mod biti (mode bit) ile donanımın kendisi denetler.



Neden önemli?

- **Ayrıcalıklı komutlar** (aygıtla konuşmak, bellek korumasını değiştirmek, işlemciyi durdurmak) yalnızca çekirdek modunda çalışır.
- Bir kullanıcı programı bunlardan birini denerse donanım işletim sistemine tuzak (trap) üretir; sistem genellikle programı sonlandırır.
- Böylece hatalı ya da kötü niyetli bir uygulama makineyi ele geçiremez.
- x86 işlemcilerde modlara halka (ring) denir: Linux ve Windows halka 0 (çekirdek) ile halka 3'ü (kullanıcı) kullanır.

UYGULAMA

Örnek 1: Çekirdeği Çağıran Bir Program

```c
/* hello_syscall.c -- aynı mesaj, iki yolla yazdırılır.
 * Derleme:  gcc hello_syscall.c -o hello_syscall
 * Çalıştır: ./hello_syscall
 * İzle:     strace ./hello_syscall     (yalnızca Linux)
 */
#include <stdio.h>  /* printf: kütüphane (kullanıcı modu) */
#include <unistd.h> /* write:  sistem çağrısı sarmalayıcısı */

int main(void)
{
    /* 1) Kütüphane çağrısı: printf metni kullanıcı
     *    modunda biçimlendirir, sonra write() ile
     *    çekirdeğe yazdırır.                    */
    printf("Hello from printf!\n");
    fflush(stdout);

    /* 2) Doğrudan sistem çağrısı: dosya tanımlayıcısı
     *    1 = ekran (standart çıktı), 20 = bayt sayısı. */
    write(1, "Hello from write()!\n", 20);

    return 0;
}
```

Ne oluyor?

- printf() bir kütüphane fonksiyonudur: metni kullanıcı modunda biçimlendirir.
- write() bir sistem çağrısıdır: çekirdek moduna geçer ve baytları ekrana çekirdek gönderir.
- strace, bir programın yaptığı tüm sistem çağrılarını gösterir:

```
$ strace ./hello_syscall
write(1, "Hello from printf!\n", 19) = 19
write(1, "Hello from write()!\n", 20) = 20
exit_group(0)                       = ?
```

Örnek 2: Çekirdeğe Bir Gidiş Ne Kadar Pahalı?

```c
#define N 1000000L

long plain_function(long x) { return x + 1; }

int main(void)
{
    volatile long sink = 0;
    double t0 = seconds();
    for (long i = 0; i < N; i++)
        sink += plain_function(i);   /* kullanıcı modu */
    double t1 = seconds();
    for (long i = 0; i < N; i++)
        sink += syscall(SYS_getpid); /* çekirdeğe gidiş */
    double t2 = seconds();

    printf("ordinary function call : %6.1f ns\n",
           (t1 - t0) / N * 1e9);
    printf("system call (getpid)   : %6.1f ns\n",
           (t2 - t1) / N * 1e9);
}
/* programın tamamı: code/syscall_cost.c */
```

Deney

- Sıradan bir fonksiyonu bir milyon kez çağır.
- Sonra bir milyon küçük sistem çağrısı (getpid) yap.
- Çağrı başına ortalama süreleri karşılaştır.
- Kesin sayılar makinenize bağlıdır; asıl ders orandadır.

```
$ ./syscall_cost
ordinary function call :   1.5 ns
system call (getpid)   : 116.1 ns
system call is about 76 times slower
```

ARA

10 Dakika Mola

Sırada: işletim sistemleri nasıl inşa edilir? MS-DOS'tan mikroçekirdeklere.

BÖLÜM 4

İşletim Sistemi Yapıları

Aynı hedefler, çok farklı binalar.

Yapı Neden Önemli?

Linux çekirdeği 30 milyonu aşkın kod satırından oluşur. Bu kadar büyük bir şeyi nasıl düzenlersiniz?

Basit

Neredeyse hiç yapı yok (MS-DOS).

Monolitik

Tek büyük bir çekirdek programı (UNIX, Linux).

Katmanlı

Katı katmanlar; her biri yalnızca altındakini kullanır (THE).

Mikroçekirdek

Küçücük bir çekirdek; hizmetler kullanıcı programı olarak çalışır (Mach, MINIX 3, QNX).

Modüler

Bir temel çekirdek ve yüklenebilir modüller (Linux).

Hibrit

Gerçek sistemler bunları karıştırır (Windows, macOS, Android).

YAPI 1

Basit Yapı: MS-DOS

1981'deki IBM PC'nin çok küçük belleğine sığmak için yazıldı; hiçbir zaman gerçekten katmanlı tasarlanmadı.



Uygulamalar katmanları atlayıp donanıma doğrudan erişebiliyordu.

Avantajlar

- Çok küçük ve hızlı.
- Sınırlı donanım için yazması kolay.

Dezavantajlar

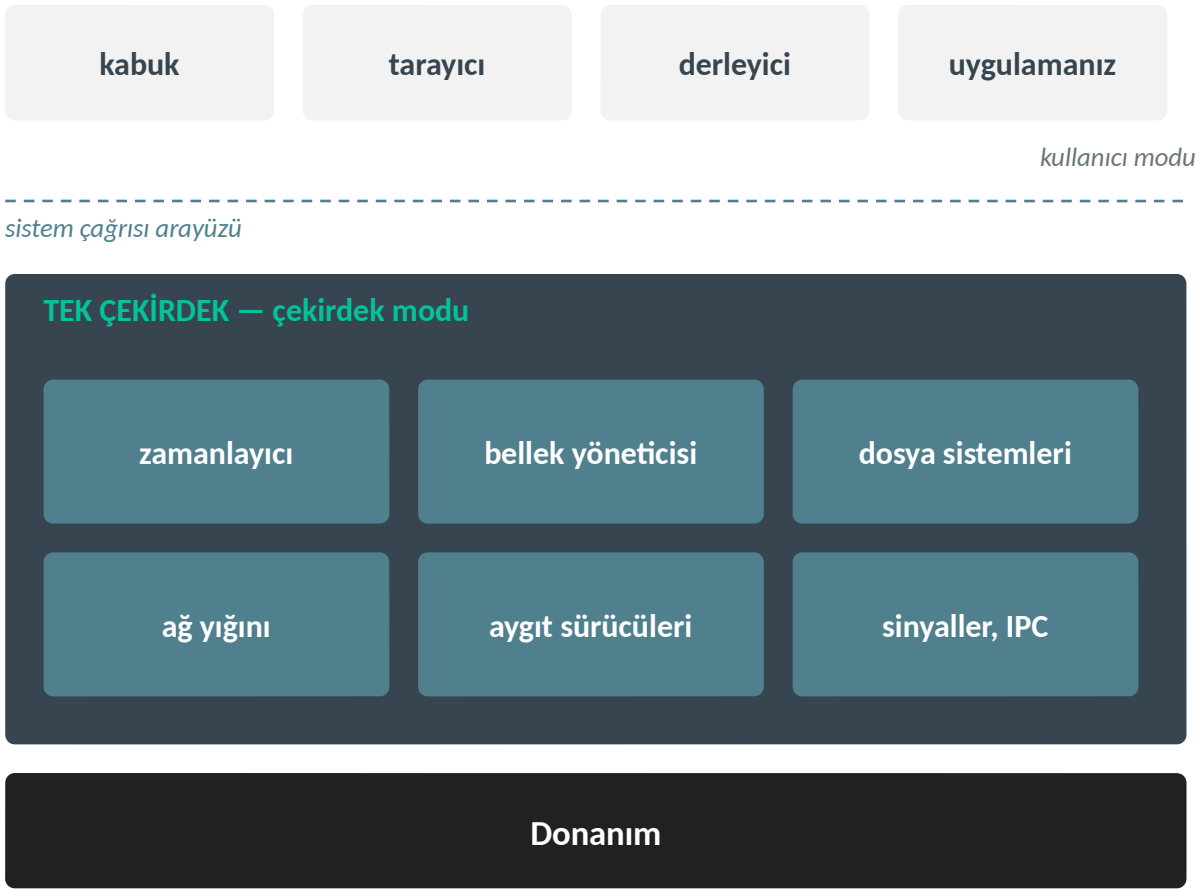
- Koruma yok: tek hatalı program tüm sistemi çökertebilirdi.
- Intel 8088'de ikili mod yoktu; işletim sistemi buna engel olamazdı.

Ders: koruma, donanım desteği (ikili mod) ve bunu kullanan bir işletim sistemi gerektirir.

YAPI 2

Monolitik Çekirdek: UNIX ve Linux

Tüm işletim sistemi hizmetleri, çekirdek modunda tek bir adres uzayında çalışan büyük bir programdadır.



Avantajlar

- Hızlı: hizmetler birbirini sıradan fonksiyon çağrıları gibi doğrudan çağırır.
- Sistem çağrısı başına yalnızca iki mod geçişi.

Dezavantajlar

- Devasa ve bakımı zor.
- Herhangi bir sürücüdeki hata bütün çekirdeği çökertebilir ("çekirdek paniği", kernel panic).

Örnekler: özgün UNIX (1970'ler), Linux, geleneksel BSD.

YAPI 3

Katmanlı Yaklaşım

İşletim sistemi katmanlara bölünür; N. katman yalnızca altındaki katmanları kullanabilir. 0. katman donanımdır.



Dijkstra'nın THE sistemine (1968) gevşek biçimde dayanır.

Avantajlar

- Katman katman inşa etmek ve hata ayıklamak kolay.
- Her katman ayrı ayrı doğrulanabilir.

Dezavantajlar

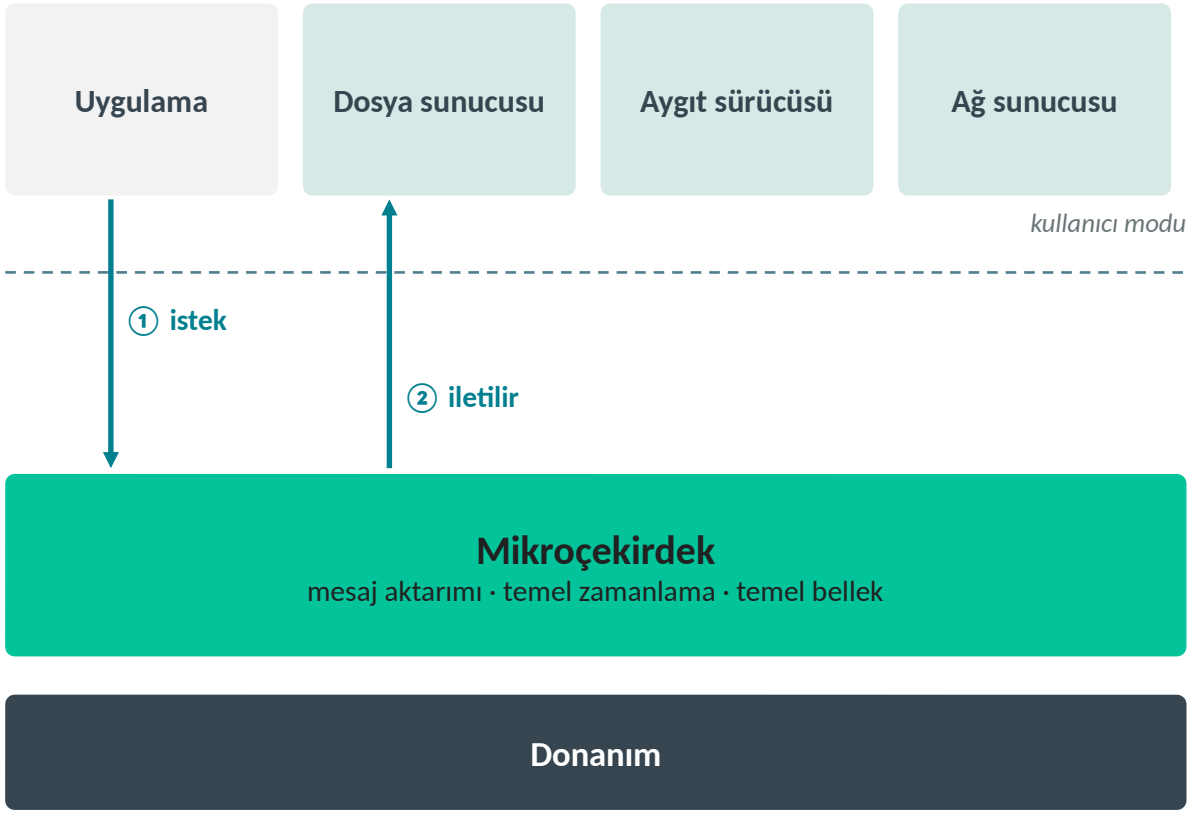
- Sırayı belirlemek zor (bellek mi zamanlayıcıya muhtaç, yoksa tersi mi?).
- Her istek birçok katmandan geçer → yavaş.

Saf katmanlı yapı bugün nadirdir, ama fikir her işletim sisteminin içinde yaşar (ör. ağ yığını katmanlıdır).

YAPI 4

Mikroçekirdek: Çekirdeği Küçücük Tut

Çekirdek modunda yalnızca asgari olan kalır; geri kalan her şey sıradan kullanıcı modu sunucuları olarak çalışır.



Uygulama dosya sunucusunu hiçbir zaman doğrudan çağırmaz: mesajlar çekirdek üzerinden gider.

Avantajlar

- Güvenilir: çöken sürücü yalnızca çöken bir programdır; yeniden başlatılır.
- Güvenli, kolay genişletilir; küçük çekirdek kolay doğrulanır.

Dezavantajlar

- Daha yavaş: her hizmet isteği ek mesajlar ve mod geçişleri gerektirir.

Örnekler: Mach (1980'ler), MINIX 3, QNX (otomobiller, tıbbi cihazlar), seL4 (matematiksel olarak doğrulanmış).

Çözümlü Örnek: Mikroçekirdeğin Bedeli

Tek bir dosya okumasında mod geçişlerini sayalım

- **Monolitik:** uygulama → çekirdek → uygulama. 2 geçiş.
- **Mikroçekirdek:** uygulama → çekirdek → dosya sunucusu → çekirdek → uygulama. 4 geçiş ve 2 mesaj kopyası.
- Varsayalım (temsilî sayılar): bir geçiş = 0,1 µs, bir mesaj kopyası = 0,15 µs, yararlı iş = 1 µs.

```
Monolitik:
  T = 2(0.1) + 1.0          = 1.2 µs

Mikroçekirdek:
  T = 4(0.1) + 2(0.15) + 1.0 = 1.7 µs

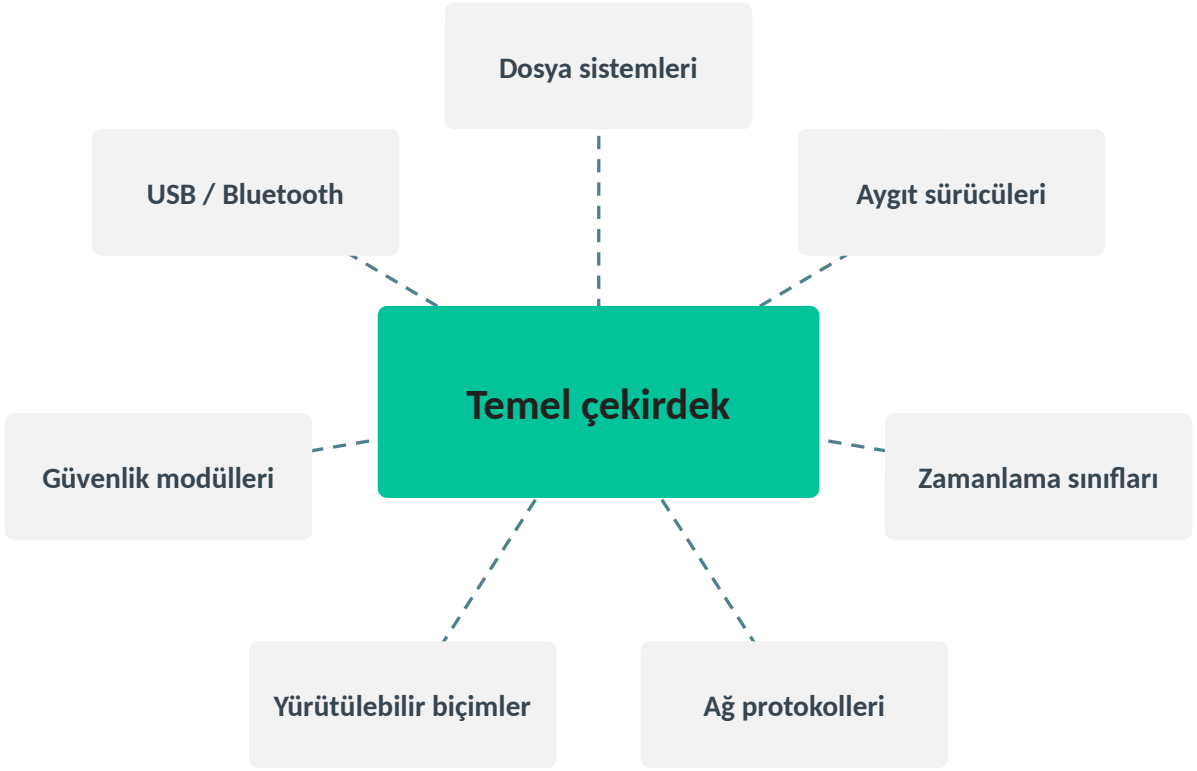
Yavaşlama: 1.7 / 1.2 ≈ 1.42  (%42)
```

Kısa ve sık işlemlerde ek yük baskındır. Uzun işlemlerde (büyük bir dosyayı okumak) pek fark etmez. L4 gibi modern mikroçekirdekler mesajları ucuzlatmak için çok çalışır.

YAPI 5

Modüler Çekirdek: Yüklenebilir Çekirdek Modülleri

Bir temel çekirdek ve sistem çalışırken eklenip çıkarılabilen modüller.



Modüller gerektiğinde yüklenir ve çalışan çekirdeğe bağlanır.

Avantajlar

- Monolitik gibi: modüller çekirdek modunda; çağrılar hızlı.
- Mikroçekirdek gibi esnek: sürücü gerektiğinde yüklenir.

Dezavantajlar

- Hatalı bir modül yine çekirdek modunda çalışır ve sistemi çökertebilir.

Örnekler: Linux (lsmod, insmod, rmmod), Solaris, modern BSD'ler. Günümüzün en yaygın tasarımı.

UYGULAMA

Örnek 3: C ile Yüklenebilir Bir Çekirdek Modülü

```c
#include <linux/init.h>
#include <linux/module.h>
#include <linux/kernel.h>

MODULE_LICENSE("GPL");
MODULE_AUTHOR("MBLP131");
MODULE_DESCRIPTION("Hello-world kernel module");

static int __init hello_init(void)   /* insmod'da çalışır */
{
    pr_info("hello_module: loaded into the kernel\n");
    return 0;                        /* 0 = başarı */
}

static void __exit hello_exit(void)  /* rmmod'da çalışır */
{
    pr_info("hello_module: removed from the kernel\n");
}

module_init(hello_init);
module_exit(hello_exit);
```

Nasıl kullanılır? (Linux)

- Çekirdeğin kendi derleme sistemiyle derlenir (code klasöründeki Makefile).
- pr_info() ekrana değil, çekirdek günlüğüne yazar.
- Bunu yalnızca bir sanal makinede deneyin.

```
$ make
$ sudo insmod hello_module.ko
$ lsmod | grep hello
$ sudo dmesg | tail -1
hello_module: loaded into the kernel
$ sudo rmmod hello_module
```

YAPI 6

Hibrit Sistemler: Gerçek İşletim Sistemleri Ne Yapar?

Saf tasarımlar nadirdir. Gerçek sistemler başarım, güvenlik ve kolaylık için birkaç yapıdan ödünç alır.

Windows (NT ailesi)

- Mikroçekirdek fikirleriyle (ayrı alt sistemler) tasarlandı.
- Hız için hizmetlerin çoğu (grafik dahil) çekirdek modunda çalışır.
- Modüller gibi yüklenebilir sürücüler.

macOS ve iOS

- "XNU" çekirdeği = Mach mikroçekirdeği + BSD UNIX parçaları, tek çekirdek olarak derlenmiş.
- Sürücüler yüklenebilir çekirdek uzantıları olarak.

Android

- Değiştirilmiş bir Linux çekirdeği (monolitik + modüller).
- Üstünde: donanım soyutlama katmanı, yerel kütüphaneler ve uygulamalar için Android çalışma zamanı.

1992: Andrew Tanenbaum (MINIX, mikroçekirdek), monolitik olduğu için Linus Torvalds'a "Linux eskimiştir" dedi. Otuz yıl sonra iki tasarım da her gün kullanılıyor.

ÖZET

Yapılar Yan Yana

Yapı	Çekirdek modunda	Hız	Güvenilirlik	Örnek
Basit	her şey (koruma yok)	yüksek	çok düşük	MS-DOS
Monolitik	tüm sistem hizmetleri	yüksek	orta	UNIX, Linux
Katmanlı	tüm katmanlar	düşük	yüksek	THE (1968)
Mikroçekirdek	yalnızca küçük çekirdek	daha düşük	yüksek	MINIX 3, QNX, seL4
Modüler	temel + yüklenen modüller	yüksek	orta	Linux, Solaris
Hibrit	değişir	yüksek	orta–yüksek	Windows, macOS, Android

Tekrarlanan ödünleşim: eklediğiniz her sınır (katman, mesaj, mod geçişi) güvenilirlik kazandırır, hızdan götürür.

KENDİNİZ DENEYİN

Kendi İşletim Sisteminizin İçine Bakın

```
# Linux (laboratuvar, WSL ya da sanal makine)
$ uname -r          # çekirdek sürümü nedir?
$ lsmod | head      # hangi modüller yüklü?
$ strace -c ls      # ls'in sistem çağrılarını say

# Windows
> winver            # hangi Windows derlemesi?
> driverquery       # kurulu sürücüleri listele
# Görev Yöneticisi > Performans: işlemci
# süresinin "kullanıcı" ve "çekirdek" payı
```

Yanıtlanacak sorular

- Makinenizde kaç çekirdek modülü yüklü?
- ls en sık hangi sistem çağrısını kullanıyor?
- ls gibi basit bir komut kabaca kaç sistem çağrısı yapar? (Test makinemizde: 76.)
- Görev Yöneticisi'ni açın ve çekirdek modunda çok zaman geçiren bir program bulun. Bunun nedeni ne olabilir?

KAPANIŞ

2. Hafta: Temel Çıkarımlar

- **Tanım:** işletim sistemi donanımı yönetir ve programlara daha basit, daha güvenli bir makine sunar; bellekte sürekli bulunan özüne çekirdek denir.
- **İki bakış:** genişletilmiş makine (soyutlama, yukarıdan aşağıya) ve kaynak yöneticisi (çoğullama, aşağıdan yukarıya).
- **Üç hedef:** kullanım kolaylığı, verimlilik, gelişebilirlik; kişisel bilgisayar, sunucu, telefon ve gömülü sistemlerde farklı ağırlıklarla.
- **Dört bileşen:** donanım, işletim sistemi, uygulama programları, kullanıcılar.
- **Koruma donanım ister:** ikili mod (kullanıcı/çekirdek) ve kesmeler; bir sistem çağrısı bir fonksiyon çağrısından çok daha pahalıdır.
- **Yapılar:** basit, monolitik, katmanlı, mikroçekirdek, modüler, hibrit; her biri hız ile güvenilirlik arasında farklı bir uzlaşma.

KISA SINAV

Kendinizi Sınayın

1. İşletim sisteminin yaptığı bir zaman çoğullama ve bir alan çoğullama örneği verin.

2. MS-DOS'ta bir programdaki hata neden bütün bilgisayarı çökertebiliyordu da Linux'ta çökertemiyor?

3. Bir sistem çağrısı 100 ns, bir fonksiyon çağrısı 1,5 ns sürüyor. 10^6 sistem çağrısı, 10^6 fonksiyon çağrısına göre ne kadar ek yük getirir?

4. Mikroçekirdeğin monolitik çekirdeğe göre bir avantajını ve bir dezavantajını söyleyin.

GELECEK HAFTA

Hizmetler, Sistem Çağrıları ve Süreçler

3. haftanın konuları:

- İşletim sisteminin sunduğu hizmetler
- Ayrıntılı sistem çağrıları: türleri, parametreleri, sistem çağrısı tablosu
- Süreç (process) kavramı: yürütülmekte olan program
- Süreç durumları: yeni, hazır, çalışıyor, bekliyor, sonlandı

SORULAR?

Tartışma

Dersten sonra sorularınız olursa dersin Telegram kanalında sorun — yanıttan herkes yararlanır.

Dr. Öğr. Üyesi Hasan Oğuz